

Kódklón detekció pipeline

Adatelőkészítés

A legnagyobb hangsúlyt ismételten a tanító adatbázis tisztaságára helyeztük. A modell tanításához szükségünk volt arra az információra, hogy két kód egymás klónja és arra is, hogyha két kód nem klónja egymásnak. Ehhez a modellt meg kellett tanítsuk eldönteni ezt a kérdés olyan bejegyzésekkel, amelyek tartalmaznak két kódot és azt a jelzőt, hogy ők egymás klónjai – e.

Az adathalmaz készítése során szándékosan a Type-3 és Type-4 klónokra fókuszáltam. A Type-1 és Type-2 klónok detektálása hagyományos, alacsony számítási igényű módszerekkel is nagy pontossággal megoldható. A GraphCodeBERT és a Cross-Encoder architektúra alkalmazását éppen a szemantikus (Type-4) hasonlóságok felismerésének nehézsége indokolja. Ezért megpróbáltam minél inkább kiszűrni azt, hogy a teljesen ugyanolyan kódokat (Type-1) megtanulja a modell. Az alap feltételezés az, hogyha a modell képes a Type-4 klónok megtalálására, akkor a Type-1 kódokat már meg kell tudnia találni.

scripts/build_clone_pairs.py

Az adathalmaz egy script segítségével készült, de hatékonyabb eredményt lehet elérni handpicked – dataset használatával. Ez a modul felelős a nyers JSONL adatokból (függvények) tanító párok (Code A, Code B, Label) generálásáért. Az alapötlete a megoldásnak az, hogy valamilyen szempont alapján csoportosítsuk a kódokat és az összetartozók egymás klónjai lesznek, míg véletlenszerűen választunk egy-egy kódot a csoporton kívül és azt berakjuk negatívként.

A probléma ezzel az volt, hogy a nyers adatokban az idx (projekt + függvéynév) alapján történt a csoportosítás. Azonban az Erlang nyelvben gyakoriak a szabványos callback függvények (pl. start/2, init/1), amelyek neve azonos, de tartalmuk projektenként teljesen eltérő. Ha ezeket vakon klónnak jelöljük, a modell hamis összefüggéseket tanul meg.

A megoldást végül az nyújtotta, hogy bevezettem egy kétlépcsős szűrési mechanizmust és a negatívok mintavételezését is szigorítottam. Még ha két függvény neve azonos is (idx egyezés), csak akkor tekintjük őket pozitív párnak (Label: 1), ha a kódjuk szavainak (tokenjeinek) átfedése eléri egy küszöbértéket (pl. 40%). Erre egy népszerű matematikai formulát a Jaccard – similarity-t használtam:

```
def jaccard_similarity(code_a, code_b):  
    toks_a = get_tokens(code_a) # set("start", "gen_server", "ok"...)  
    toks_b = get_tokens(code_b)  
    return len(toks_a & toks_b) / len(toks_a | toks_b)
```

```
if sim < min_pos_sim:
    continue
```

A negatívok mintavételezésénél az volt a tervezői gondolat, hogy ha 5 soros kódot párosítanánk egy 100 sorossal, a modell megtanulná azt az olcsó trükköt, hogy "ha nagy a hosszkülönbség -> nem klón". A hasonló hosszúságú negatív párok kényszerítik a modellt, hogy a szemantikát vizsgálja, ne a metrikákat. Ezért a negatív párok (Label: 0) generálásakor nem teljesen véletlenszerűen választunk. Olyan kódokat párosítunk össze, amelyek hossza hasonló.

```
if length_match:
    len_a = len(a["_code_norm"])
    len_b = len(b["_code_norm"])
    # Kerüljük a 0-val osztást
    ratio = len_b / max(1, len_a)
    if not (length_match[0] <= ratio <= length_match[1]):
        continue
```

Adatbetöltés és Reprezentáció

datasets/clone_pairs_dataset.py

Ez a réteg készíti elő az adatokat úgy, hogy a nyers szöveges párok és a neurális háló által emészthető tenzorok közötti átalakítást elvégzi.

A Cross-Encoder stratégia: A legfontosabb komponens a ClonePairsCollator. Ahelyett, hogy külön-külön vektorizálnánk a két kódot (Bi-Encoder), a tokenizer segítségével egyetlen szekvenciává fűzzük össze őket.

```
toks = self.tok(
    texts_a,      # Code A
    texts_b,      # Code B
    padding=True,
    truncation=True,
    max_length=256,
    return_tensors="pt"
)
```

Ez a következő bemenetet állítja elő a modellnek: „[CLS] Code_A tokenek ... [SEP] Code_B tokenek ... [SEP]”

A Transformer háló önfigyelés mechanizmusa így egyszerre látja mindkét kódot. Minden token képes "figyelni" a másik kód összes tokenjére.

Modell Architektúra

train/model_clone.py

Felépítés:

Encoder (GraphCodeBERT): Feldolgozza az összefűzött tokensorozatot.

Pooling: Kiemeljük a [CLS] (Class) token rejtett állapotát. Ez a speciális token a tanítás során megtanulja sűríteni a teljes bemeneti szekvencia (tehát a két kód viszonyának) információját.

Dropout: 0.1-es rátával véletlenszerűen kikapcsolunk neuronokat a túlilleszkedés (overfitting) ellen.

Classifier (Linear): Egy sűrű réteg, amely a 768 dimenziós vektort 2 dimenzióra (Nem klón, Klón) képezi le.

```
def __init__(self, base_checkpoint: str, dropout: float = 0.1):
    super().__init__()
    self.config = AutoConfig.from_pretrained(base_checkpoint)
    self.encoder = AutoModel.from_pretrained(base_checkpoint)
    hidden = self.config.hidden_size
    self.dropout = nn.Dropout(dropout)
    self.classifier = nn.Linear(hidden, 2)
```

```
def forward(self, input_ids, ...):
    out = self.encoder(input_ids, ...)
    cls_vector = out.last_hidden_state[:, 0, :]
    logits = self.classifier(self.dropout(cls_vector))
    return logits
```

Tanítási Folyamat

train/train_clone_detector.py

Inicializálás

A neurális hálók inicializálása véletlenszerű súlyokkal történik. Nem töltjük be az összes adatot egyszerre a GPU-ra (mert nem férne el). A DataLoader felelős azért, hogy az adatokat batch-ekre (kisebb kötegekre, pl. 16 pár) bontsa, és véletlenszerű sorrendben (shuffle=True) adagolja a modellnek.

```
ds_train = ClonePairsDataset(args.train_file)
dl_train = DataLoader(
    ds_train,
    batch_size=args.batch_size,
    shuffle=True,      # Véletlenszerű sorrend
    num_workers=0,
    collate_fn=collator # Cross-Encoder összefűzés
)
```

A Tanulási Ciklus

A modell megkapja a tokenizált kódpárokat (`input_ids`, `attention_mask`). Ezek átfolynak a GraphCodeBERT rétegein, majd a saját osztályozó fejünkön. A kimenet a **Logit**: a modell nyers tippje. Összehasonlítjuk a modell tippjét (Logit) a valósággal (Label: 0 vagy 1). Ehhez a `CrossEntropyLoss` függvényt használjuk.

Mixed Precision Training (FP16): A `torch.cuda.amp` és `GradScaler` használatával a számítások egy részét 16-bites lebegőpontos számokkal végezzük 32-bites helyett.

```
with autocast(enabled=args.fp16):
    out = model(**batch, labels=labels)
    loss = out["loss"]

scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

Visszaterjesztés és Optimalizálás: Ez a tanulás pillanata. Kiszámoljuk a gradienseket (milyen irányba kell módosítani a súlyokat, hogy csökkenjen a hiba), majd az AdamW optimalizálóval frissítjük a modellt.

Validáció és Metrikák

Minden epoch végén megállítjuk a tanítást, és a modellt átkapcsoljuk értékelő (eval) módba. Ilyenkor a `valid_pairs.jsonl` (validációs halmaz) adatait mutatjuk neki, amiket sosem látott a tanítás során, itt számoljuk az F1-Score-t.

Early Stopping (Korai leállás): Nem futtatjuk a tanítást fixen 10 epoch-ig, ha a modell már nem javul. Figyeljük a validációs F1-score-t. Ha `patience=3` epochon keresztül nem érünk el új csúcst, a tanítás leáll, és visszaállítjuk a legjobb mentett állapotot. Ez megakadályozza, hogy a modell "bemagolja" a tanító adatokat (overfitting).

```
if val_f1 > best_f1:
    best_f1 = val_f1
    patience_counter = 0
else:
    patience_counter += 1

if patience_counter >= args.patience:
    print(f"[STOP] Early stopping triggered after {args.patience} epochs.")
    break
```